



PIT 2026 • CURSO GRATUITO

Programa de Iniciación Tecnológica

Git y GitHub

Control de Versiones Software

Sesión 1 • Fundamentos de Git y primer repositorio

Luis Rojas

Instructor del curso • Universidad Nacional de Ingeniería

Modalidad virtual (Zoom + YouTube) • 3 horas con receso

En el aula

- Pregunten sin miedo.
- Equivocarse es parte del oficio.
- Practicamos en vivo.
- Hay receso a mitad de clase (15 a 20 min).

Temas de esta sesión

- Control de versiones y por qué Git.
- Comandos básicos de terminal.
- Instalación y configuración.
- Primer repositorio local: **init**, **status**, **add**, **commit**, **log**.

Cómo se califica

- Hay **cuatro tests** a lo largo del curso.
- Hay **un examen final**.
- De los cuatro tests se elimina la nota más baja. Las tres restantes se suman al **doble** de la evaluación final.

Certificado UNI

- Digital y **opcional**.
- Promedio final ≥ 12 : certificado.
- Si no aprueba, puede haber constancia de asistencia con $\geq 75\%$ de clases.

La asistencia se abre sola en **PIT Virtual** durante el horario de clase.

Conocimientos

- Manejo básico de archivos y carpetas.
- Noción simple de terminal (o ganas de aprenderla hoy).
- Disposición a practicar y equivocarse.

Herramientas

- Computadora con Windows, macOS o Linux.
- Git instalado o listo para instalar.
- Un editor: VS Code, notepad, nano...
- Terminal: **Git Bash** (recomendado en Windows), PowerShell, Terminal de macOS o shell de Linux.

tesis_final_v2_CORREGIDA_ahora_sí.zip

tesis_final_v3_revisar_lunes.rar backup_viernes copia2

El síntoma	El riesgo	El límite
Carpetas duplicadas, archivos que se pisan y la pregunta eterna: <i>¿cuál era la buena?</i>	Un cambio malo borra horas de trabajo. No hay autor, ni fecha, ni motivo.	El correo y el pendrive no escalan cuando hay dos, cinco o cincuenta personas.

Idea clave. Ese desorden no es falta de orden personal. Es falta de un **sistema** que recuerde por nosotros.

Qué es un control de versiones

Un **sistema de control de versiones** (VCS, *version control system*) guarda la historia de un proyecto para responder tres preguntas:

¿Qué cambió?

¿Cuándo cambió?

¿Por qué y quién?

Historial

Cada cambio importante queda en una línea de tiempo, no en una carpeta suelta.

Recuperación

Si algo se rompe, se puede volver a un punto anterior sin adivinar.

Trazabilidad

Se puede auditar quién tocó qué. Eso importa en equipo y en producción.

Fuente conceptual: documentación oficial de Git (<https://git-scm.com/book>) y Wikipedia, *Control de versiones*.

Por qué Git es indispensable en proyectos reales

1

Seguridad

Recupera versiones anteriores sin depender de copias manuales ni de la memoria.

2

Orden

Cada cambio queda documentado: mensaje, autor, fecha e identificador único.

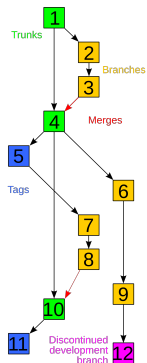
3

Colaboración

Varias personas trabajan sobre el mismo proyecto con reglas claras, no con archivos que se pisan.

Hoy Git es el estándar de facto en software, ciencia de datos, documentación y hasta en sitios web. No es una moda de programadores: es la bitácora del trabajo intelectual.

Cómo se ve un proyecto con historial



Esta figura de Wikimedia Commons muestra la idea: el proyecto no es *un* archivo, es una **secuencia de revisiones**.

Cada nodo es un estado que se puede nombrar, comparar y, si hace falta, restaurar.

File:Revision controlled project visualization.svg · Wikimedia Commons.

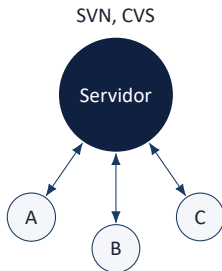
Evolución: de la copia manual a Git



- **Antes:** duplicar carpetas. Rápido el primer día, frágil al segundo.
- **CVS:** historial en un servidor. Útil, pero pesado con ramas y binarios.
- **Subversion (SVN):** modelo centralizado más sólido; un servidor es la verdad.
- **Git:** cada copia local tiene el historial completo. Rápido, ramificable, pensado para proyectos enormes.

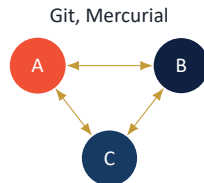
Fuentes: Wikipedia (*Concurrent Versions System*, *Apache Subversion*, *Git*) y Pro Git, cap. 1.

Modelo centralizado



El historial vive en un servidor. Sin red, el trabajo profundo se traba.

Modelo distribuido



Cada persona tiene el historial completo. Se puede confirmar cambios **sin internet**.

Ojo. Distribuido no significa *sin servidor*. Significa que el servidor no es el único dueño de la historia.

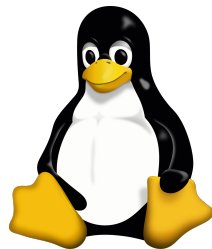
2005: por qué nació Git

El kernel de **Linux** es uno de los proyectos de software más grandes del mundo. Entre 2002 y 2005 su desarrollo se apoyó en **BitKeeper**, un sistema propietario.

En abril de 2005 se revocó la licencia gratuita que usaba la comunidad. Linus Torvalds necesitaba, en cuestión de días, una herramienta que fuera:

- **Rápida** con un árbol enorme.
- **Distribuida**, como el flujo real del kernel.
- Capaz de ramificar y fusionar sin drama.
- **Íntegra**: que la historia no se pudiera alterar en silencio.

Fuentes: Wikipedia, *Git (software)*; lista linux-kernel, anuncio de Torvalds, abril de 2005.



Tux, mascota de Linux.

Larry Ewing / Wikimedia Commons.



LinuxCon Europe, 2014.

Wikimedia Commons, CC BY 2.0.

Linus Benedict Torvalds (n. 1969) creó el kernel Linux en 1991 y Git en 2005.

No diseñó Git para una clase ni para un producto comercial. Lo diseñó porque el kernel se le iba a detener sin una herramienta a la altura.

El 3 de abril de 2005 empezó el desarrollo. El 7 de abril Git ya se hospedaba a sí mismo. En junio ya gestionaba un lanzamiento del kernel.

Desde julio de 2005 el mantenimiento lo lidera **Junio Hamano**, con cientos de colaboradores. Licencia: **GPL-2.0**.



Esta fotografía de 2004, poco antes de Git, recuerda que estamos hablando de una persona concreta, no de una marca.

Torvalds buscaba velocidad y un modelo mental simple: Git no guarda *diffs* como pieza central; guarda **instantáneas** (*snapshots*) del proyecto.

El nombre *git* es, en slang británico, un insulto suave. El propio Linus ha bromeado con ello. El software, en cambio, es extraordinariamente serio.

Foto: Linux Magazine / Wikimedia Commons, CC BY-SA 3.0.

Datos: Wikipedia EN/ES *Git*; Pro Git, «A Short History of Git».



Git es un sistema de control de versiones **distribuido**.

- Se instala en **su computadora**. No es una página web.
- Cada repositorio local contiene el historial del proyecto.
- Puede confirmar cambios (**commit**) **sin internet**.
- Sitio oficial: <https://git-scm.com> Documentación: <https://git-scm.com/doc>
- Versión estable reciente: **2.55** (junio de 2026), según Wikipedia/Wikidata.

Cinco ideas de diseño que todavía importan

- ❶ **Instantáneas, no parches sueltos.** Cada commit es una foto del proyecto.
- ❷ **Casi todo es local.** Ver historial, crear ramas y confirmar no exigen red.
- ❸ **Integridad.** Cada objeto tiene un hash (SHA-1 clásico; Git moderno también habla SHA-256). Si algo cambia, el identificador cambia.
- ❹ **Git añade; rara vez destruye.** Es difícil perder trabajo que ya está en un commit.
- ❺ **Ramas baratas.** Ramificar no es un lujo: es la forma normal de trabajar. Hoy solo veremos la idea; la práctica llega en la sesión 3.

Adaptado de Pro Git, cap. 1.3, y de la documentación oficial. No es una cita literal.

No.

Y esta es la confusión más cara del curso si no se aclara hoy.

Git

- Herramienta de línea de comandos.
- Vive en su disco.
- Controla versiones **localmente**.
- Software libre (GPL-2.0).
- Existe desde 2005.

GitHub

- Plataforma web (servicio).
- Aloja repositorios en la nube.
- Facilita colaboración, revisión y CI.
- Empresa (adquirida por Microsoft en 2018).
- Nace en 2008; **usa Git por debajo**.

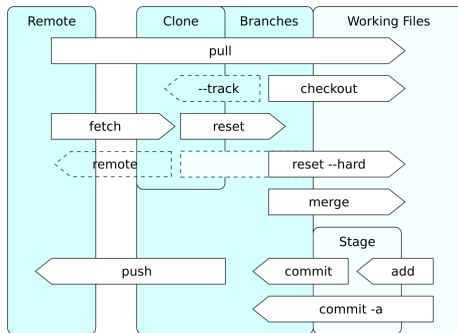
La analogía que conviene memorizar



- Git es el motor. GitHub es *un* lugar donde estacionar el proyecto.
- También existen **GitLab**, **Bitbucket**, **Codeberg**, Gitea, el servidor de su universidad...
- Si GitHub cayera mañana, Git seguiría funcionando en su laptop.

Idea clave. En las sesiones 1 a 3 dominamos Git local. En las sesiones 4 a 6 subimos ese oficio a GitHub.

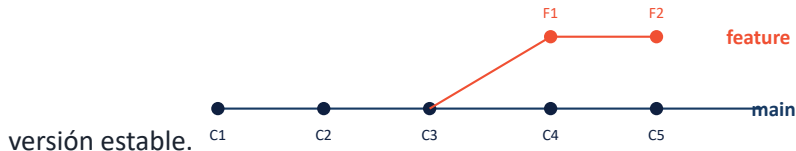
Mapa que veremos a lo largo del curso



File:Git operations.svg · Wikimedia Commons. Hoy nos quedamos a la derecha: archivos de trabajo, *stage* y repositorio local. **push/pull** llegan en la sesión 4.

Vista previa (solo la idea): qué es una rama

Una **rama** es una línea de trabajo dentro del historial. Permite experimentar sin romper la



- **main** (o **master** en repos antiguos) suele ser la línea estable.
- Hoy solo necesitamos **reconocer** el nombre que Git muestre al inicializar.
- Crear, cambiar y fusionar ramas: sesión 3.

Pausa

Estírense, tomen agua, despejen la vista del monitor.

Encargo corto durante el receso

- 1 Entren a <https://git-scm.com/downloads> y descarguen el instalador si aún no tienen Git.
- 2 En Windows, elijan **Git Bash** cuando el asistente lo ofrezca.
- 3 No configuren todavía nombre ni correo: lo hacemos juntos al volver.

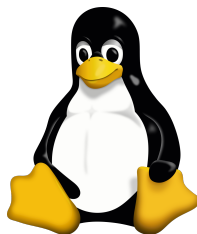
Volvemos para terminal, instalación guiada y el laboratorio del primer repositorio.

Por qué Git se trabaja en la terminal

Git tiene interfaces gráficas, y GitHub es visual. Aun así, en este curso priorizamos la terminal porque:

- Muestra el **estado real** del repositorio, sin magia oculta.
- Los mismos comandos sirven en Windows, macOS, Linux y servidores.
- Es el lenguaje que verán en documentación, ofertas laborales y DevOps.

No hace falta ser experto en Linux. Hace falta moverse con calma: carpeta actual, listar, crear, entrar, salir.



Linux + terminal son la base práctica de Git y de todo el flujo DevOps posterior.

Comandos de terminal que usaremos hoy

Comando	Equiv. PowerShell	Para qué
<code>pwd</code>	<code>Get-Location</code>	Dónde estoy.
<code>ls / ls -la</code>	<code>dir / Get-ChildItem</code>	Listar archivos (<code>-la</code> : ocultos).
<code>cd carpeta</code>	<code>Set-Location</code>	Entrar a una carpeta.
<code>cd ..</code>	<code>cd ..</code>	Subir un nivel.
<code>mkdir curso-git</code>	<code>mkdir</code>	Crear carpeta.
<code>touch notas.txt</code>	<code>ni notas.txt</code>	Crear archivo vacío.
<code>cat notas.txt</code>	<code>Get-Content</code>	Ver contenido.
<code>clear</code>	<code>clear</code>	Limpiar pantalla.

Ojo. En Windows recomendamos **Git Bash** para que los comandos coincidan con macOS/Linux y con el resto del curso.

Mini práctica: Llegar a una carpeta de trabajo

Cree una carpeta de curso *fuera* de OneDrive/Descargas si puede (menos sorpresas con archivos ocultos).

```
pwd  
mkdir -p ~/curso-git/sesion-01  
cd ~/curso-git/sesion-01  
ls -la
```

En Windows con Git Bash, ~ suele ser C:/Users/SuUsuario.

Compruebe: el prompt debe mostrar que está dentro de `sesion-01`. Si se pierde, **pwd** es la brújula.

```
git --version
```

- Si aparece algo como `git version 2.xx.x`, está instalado.
- Si dice *no se reconoce el comando / command not found*, hay que instalar o corregir el PATH.
- En Windows, cierre y abra de nuevo Git Bash después de instalar.

Dato útil

Git para Windows incluye Git Bash, Git GUI y, opcionalmente, integración con Windows Terminal. El curso se dicta con comandos de Bash.

Instalación por sistema

Windows	macOS	Linux
Descargue el instalador standalone desde https://git-scm.com . Recomendaciones del asistente: Git Bash, rama por defecto main, editor VS Code si lo tiene, <i>Checkout as-is, commit Unix-style</i> .	Homebrew: <code>brew install git</code> O las herramientas de Xcode: <code>xcode-select --install</code>	Debian/Ubuntu: <code>sudo apt install git</code> Fedora: <code>sudo dnf install git</code>

Verificación final: abra una terminal nueva y ejecute **git --version**.

Configurar identidad: nombre y correo

Git **firma** cada commit con estos datos. No es una cuenta de GitHub (eso es sesión 4). Es su identidad local.

```
git config --global user.name "Luis Rojas"  
git config --global user.email "luis.rojas@ejemplo.com"
```

- Use su nombre real o el que quiera ver en el historial.
- El correo puede ser el institucional o uno personal. Más adelante, si usan GitHub, conviene que coincida o que activen el correo privado de GitHub.
- **--global** aplica a *todos* los repositorios de este usuario en esta máquina.

Tres ajustes más que evitan dolores de cabeza

```
git config --global init.defaultBranch main
git config --global color.ui auto
git config --global core.editor "code --wait"
```

- **init.defaultBranch main**: los repos nuevos nacen en main, no en master.
- **color.ui**: la terminal se vuelve legible.
- El editor se abre cuando Git pide un mensaje largo. Si no tiene VS Code, use notepad (Windows) o nano.

```
git config --list
git config user.name
git config user.email
```

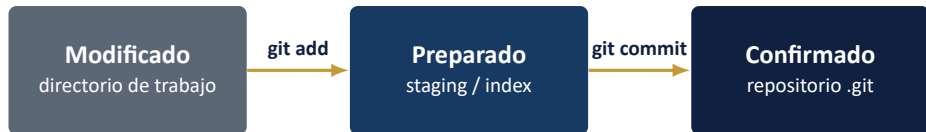
Dónde vive esa configuración

Git lee valores en cascada, de más general a más específico:

- ❶ **Sistema:** todos los usuarios de la máquina (`git config --system`).
- ❷ **Global:** su usuario. En Linux/macOS suele ser `~/.gitconfig`. En Windows: `C:\Users\Usted\.gitconfig`.
- ❸ **Local:** solo este repositorio, archivo `.git/config`. Gana sobre lo global.

Idea clave. Si un dato *no cuadra*, pregunte: ¿lo configuró en global o solo dentro de un repo?

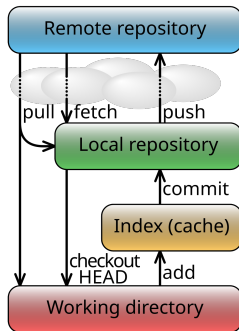
Los tres estados (el mapa mental de hoy)



- ❶ Usted **edita** archivos (la carpeta que ve).
- ❷ Elige qué entra en la próxima foto: **git add**.
- ❸ Congela esa foto en la historia: **git commit**.

Idea clave. Staging no es burocracia: es el derecho a *no* meter el archivo del almuerzo en el mismo commit que el arreglo del login.

Flujo local simplificado



File:Git data flow simplified.svg · Wikimedia Commons. Working directory → index → HEAD. El recuadro *remote* se deja para la sesión 4.

Paso 1: `git init`

```
mkdir mi-primer-repo  
cd mi-primer-repo  
git init
```

- Crea la carpeta oculta `.git`: ahí vive toda la historia.
- **No borre** `.git` *a menos que* quiera destruir el repositorio.
- Un repo Git es, en el fondo, esa carpeta más sus archivos de trabajo.

Ojo. Inicialice Git *dentro* del proyecto, no en `C:\` ni en `/home`. Un **`git init`** en el disco entero es un clásico accidente de principiante.

Paso 2: `git status` (su brújula)

```
git status
```

Antes de cualquier comando raro, **`git status`**. Responde:

- En qué rama está.
- Qué archivos son nuevos (*untracked*).
- Qué está modificado y qué ya está en staging.
- Si el árbol está limpio: *nothing to commit, working tree clean*.

Hábito profesional

Status → actúe → status otra vez. Así se entrena el ojo.

Paso 3: crear un archivo y prepararlo

Cree README.md (con el editor o con la terminal):

```
echo "# Mi primer repositorio" > README.md  
echo "Curso PIT 2026 - Sesión 1" >> README.md  
git status  
git add README.md  
git status
```

- **git add README.md** prepara ese archivo.
- **git add .** prepara todo lo del directorio actual. Úselo cuando sepa qué hay.

Paso 4: `git commit` (la foto)

Un **commit** es una instantánea confirmada: autor, fecha, mensaje e identificador único (hash).

```
git commit -m "Agrega README inicial del curso"
```

Buenos mensajes

- Modo imperativo: *Agrega, Corrige, Actualiza.*
- Un commit = un cambio lógico.

Evite

- asdf, fix, cambios, wip.
- Mezclar login + CSS + tesis en uno solo.

Regla práctica: si no puede decir el cambio en una frase, el commit es demasiado grande.

Paso 5: `git log` (leer el pasado)

```
git log
git log --oneline
git log --oneline --graph --all
```

- **git log** muestra autor, fecha, hash y mensaje.
- **--oneline** es la vista de trabajo diario.
- El hash identifica **ese** instante para siempre.

HEAD es un puntero a *dónde está usted ahora*, normalmente el último commit de la rama activa. Hoy basta con esa intuición.

Objetivo: un repositorio con al menos dos commits y un log legible.

- ➊ `mkdir notas-pit && cd notas-pit && git init`
- ➋ Cree `README.md` con su nombre y la frase: *Sesión 1*.
- ➌ `git add README.md` y commit: Agrega `README` de notas del curso.
- ➍ Cree `comandos.txt` con tres comandos que aprendió hoy.
- ➎ Segundo commit: Agrega lista personal de comandos.
- ➏ Ejecute `git log --online` y copie el resultado (pantallazo para el aula virtual).

Si algo falla: `pwd`, `ls -la`, `git status`. Esos tres resuelven el 80 % de los sustos de hoy.

Errores frecuentes de la primera hora práctica

- **Commit sin add.** Git responde *nothing added to commit*. Falta staging.
- **Mensaje vacío / editor que no cierra.** Configure **core.editor** o use **-m**.
- **Nombre y correo en blanco.** **git config --global** otra vez.
- **Inicializó en la carpeta equivocada.** **pwd**. Si creó **.git** donde no debía y *aún no hay commits valiosos*, se puede borrar esa carpeta **.git** con cuidado.
- **No ve .git.** En Bash: **ls -la**. En el Explorador de Windows: ver archivos ocultos.

Ojo. No use todavía **git reset --hard** ni borre archivos a ciegas. Si el trabajo no está en un commit, se puede perder.

Un extra breve: .gitignore

.gitignore le dice a Git qué **no** rastrear: dependencias, logs, secretos, binarios.

```
# ejemplo minimo - no lo copie sin entenderlo
*.log
.env
__pycache__/
node_modules/
```

- Nunca suba contraseñas, claves SSH ni archivos .env.
- Hoy basta saber que existe. En la sesión 4 lo usaremos al publicar.

Qué no haremos todavía (a propósito)

Para no mezclar oficios:

- No creamos cuenta de GitHub ni hacemos **git push**.
- No clonamos ni hacemos fork.
- No resolvemos conflictos ni hacemos merge real.
- No reescribimos historia con reset duro.

Eso no es que *falte* Git. Es que un cimiento limpio vale más que diez comandos a medias.

- ① El control de versiones responde *qué, cuándo, quién y por qué*.
- ② Git nació en 2005 para el kernel Linux, después de BitKeeper.
- ③ **Git** $\neq$ **GitHub**. Motor local frente a plataforma web.
- ④ La terminal es el volante; **pwd** y **git status** son el tablero.
- ⑤ Identidad: **user.name** y **user.email**.
- ⑥ Flujo local: editar $\rightarrow$ **add** $\rightarrow$ **commit** $\rightarrow$ **log**.

Hoja de comandos de hoy (guárdela)

Tarea para la sesión 2

- ❶ Deje Git instalado y configurado en *la* computadora con la que seguirá el curso.
- ❷ Complete el laboratorio de notas-pit si no alcanzó en clase.
- ❸ Lea, sin implementar todavía, Pro Git en español, cap. 1:
<https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>
- ❹ Traiga una duda escrita. Las dudas concretas adelantan más que *no me funciona*.

Próxima sesión: cómo Git registra cambios, **git diff**, explorar el log con criterio y recuperar trabajo sin pánico.

- Documentación oficial: <https://git-scm.com/doc>
- Libro libre *Pro Git* (Chacon y Straub): <https://git-scm.com/book/es/v2>
- Wikipedia ES: <https://es.wikipedia.org/wiki/Git>
- Wikipedia EN (historia BitKeeper, fechas):
[https://en.wikipedia.org/wiki/Git_\(software\)](https://en.wikipedia.org/wiki/Git_(software))
- Descarga e instaladores: <https://git-scm.com/downloads>
- Guía Ubuntu CLI (apoyo de terminal): https://assets.ubuntu.com/v1/d00791ae-ubuntu_cli_cheat_sheet_2025.pdf

Las diapositivas son material original del curso PIT 2026. Los hechos históricos se contrastaron con Wikipedia y Pro Git; las imágenes tienen ficha de atribución en `imagenes/ATRIBUCION.txt`.

Gracias.

Luis Rojas

Sesión 1 · PIT 2026 · Universidad Nacional de Ingeniería

¿Preguntas de último minuto? El chat y el aula virtual siguen abiertos.